

CSN 342

Deep Learning

Course Instructor/Faculty: Dr. Vyomika Singh

UNIT 2: Feed Forward Networks (Topics covered in this PPT)

Multilayer Perceptron (MLP) and activation functions. Logistic Regression, Learning parameters, Loss functions, **Empirical Risk Minimization, Gradient Descent and Backpropagation algorithm**. Regularization techniques. Difficulty of training deep neural networks, Greedy layer wise training.

CSN 342

Deep Learning

Gradient Descent: Definition

According to various literature sources:

Gradient Descent is:

1. An optimization technique/algorithm.
2. Mostly used in supervised machine learning models and deep learning.
3. Also called as **first order optimization algorithm**.
4. One of the most used algorithms for optimization of parameters in ML models.

The meaning of Gradient Descent:

1. The meaning of Gradient: first order derivative/ slope of a curve.
2. The meaning of descent: movement to a lower point.
3. **The algorithm thus makes use of the gradient/slope to reach the minimum/ lowest point of a loss/cost function.**

CSN 342

Deep Learning

Gradient Descent

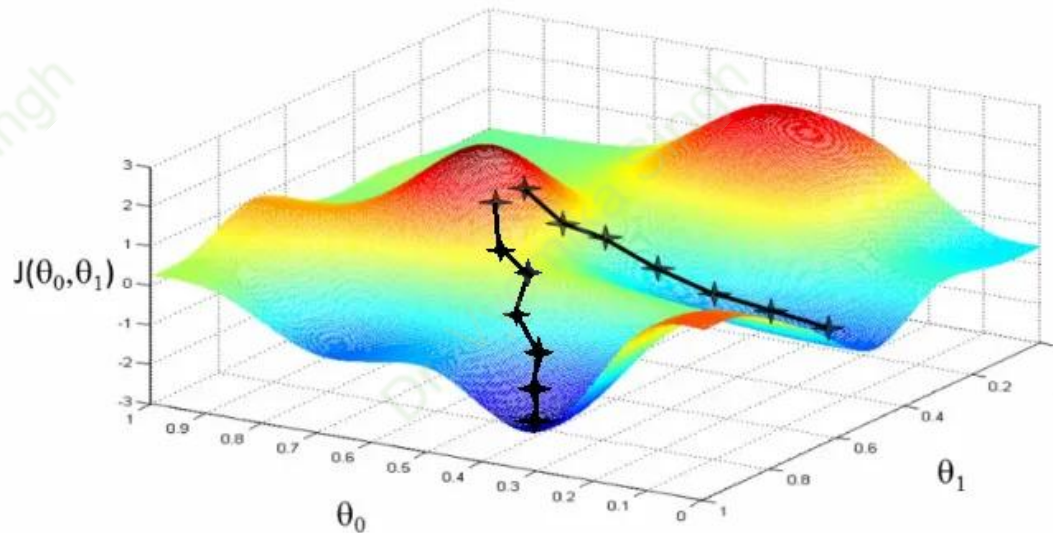
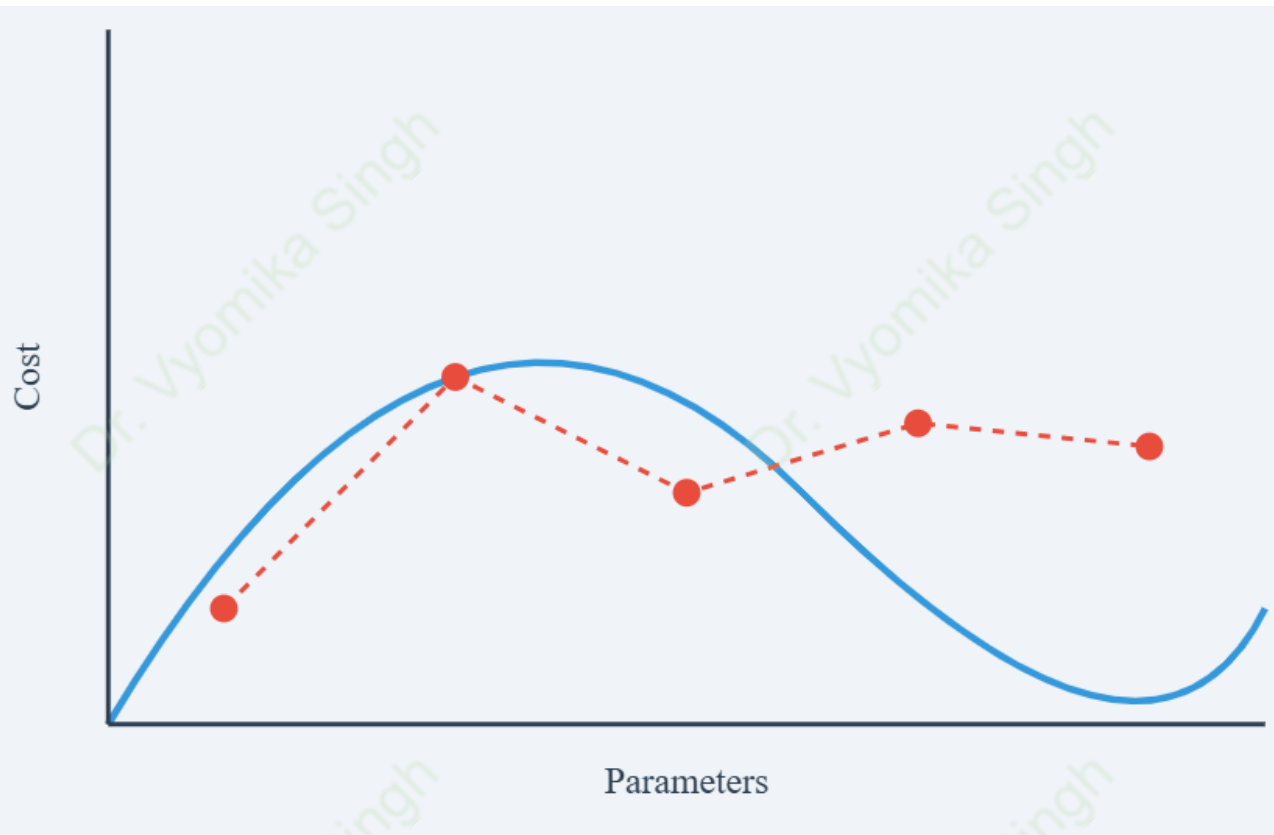


Image: Gradient Descent is an algorithm to find the minimum of a function in N dimensions. However, some functions are complicated and show multiple minima.

CSN 342

Deep Learning

Gradient Descent: Concept



Concept: The cost function or loss function measures how well the model's predictions match the actual data. By iteratively adjusting the parameters in the direction that reduces the cost function, the model improves its accuracy.

CSN 342

Deep Learning

Gradient Descent: Requirements

Function $f(x)$ needs to be:

Differentiable

Convex

CSN 342

Deep Learning

Gradient Descent: Requirements

Differentiable: Being differentiable means the function should have a derivative at each point in its domain

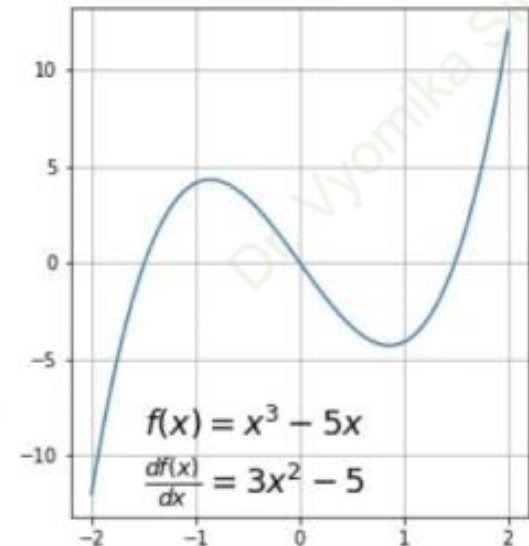
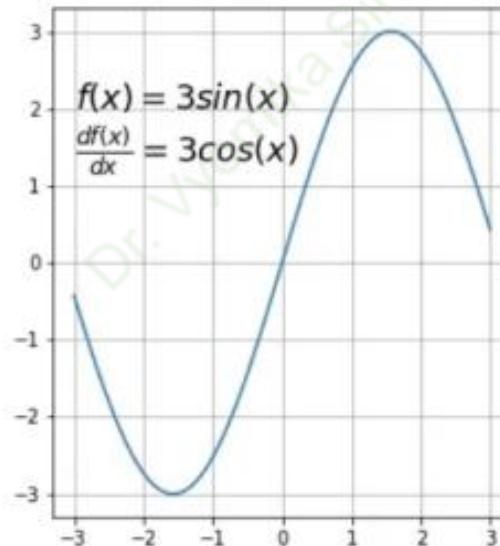
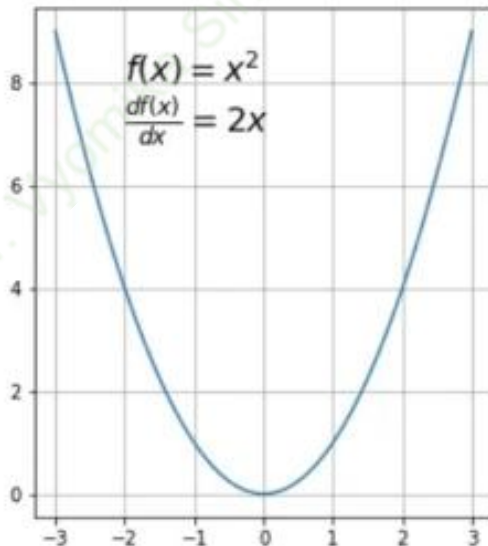


Figure: Examples of differentiable functions.[Kwiatkowski 2022]

CSN 342

Deep Learning

Gradient Descent: Requirements

Not all functions satisfy this condition. Some examples of functions not satisfying this condition is given below in Fig. If the function is not differentiable at all points, there may arise a circumstance where the gradient algorithm is not able to **iterate the values as there is no gradient to work with.**

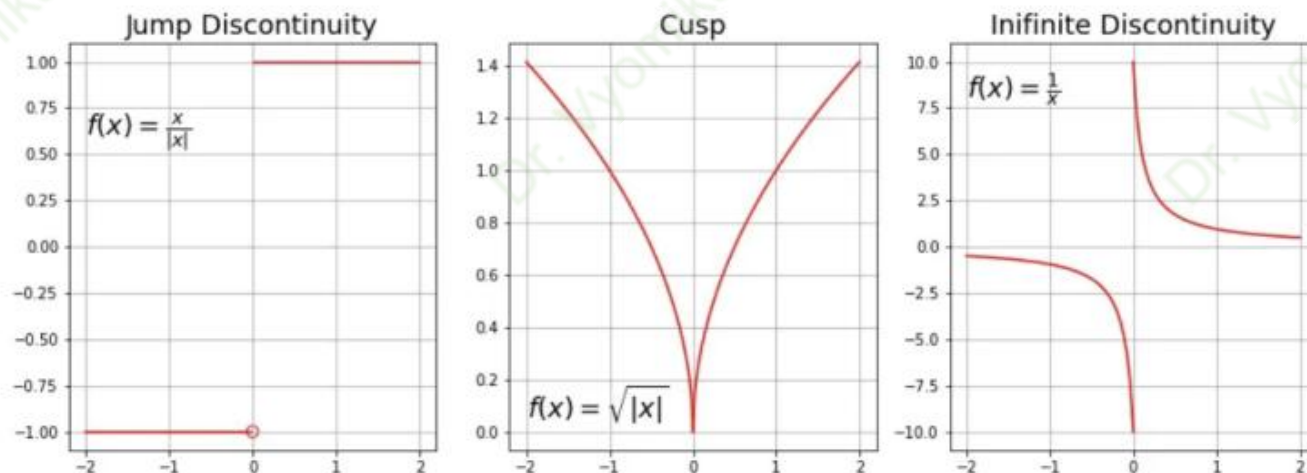


Figure: Examples of non-differentiable functions.[Kwiatkowski 2022]

CSN 342

Deep Learning

Gradient Descent: Requirements

Convex function: A convex function is one in which if any two points are connected on the curve, the line segment lies on or above the curve. If a function is non-convex, we do not have a surety that the gradient descent algorithm would give us the local minima or the global minima as the result.

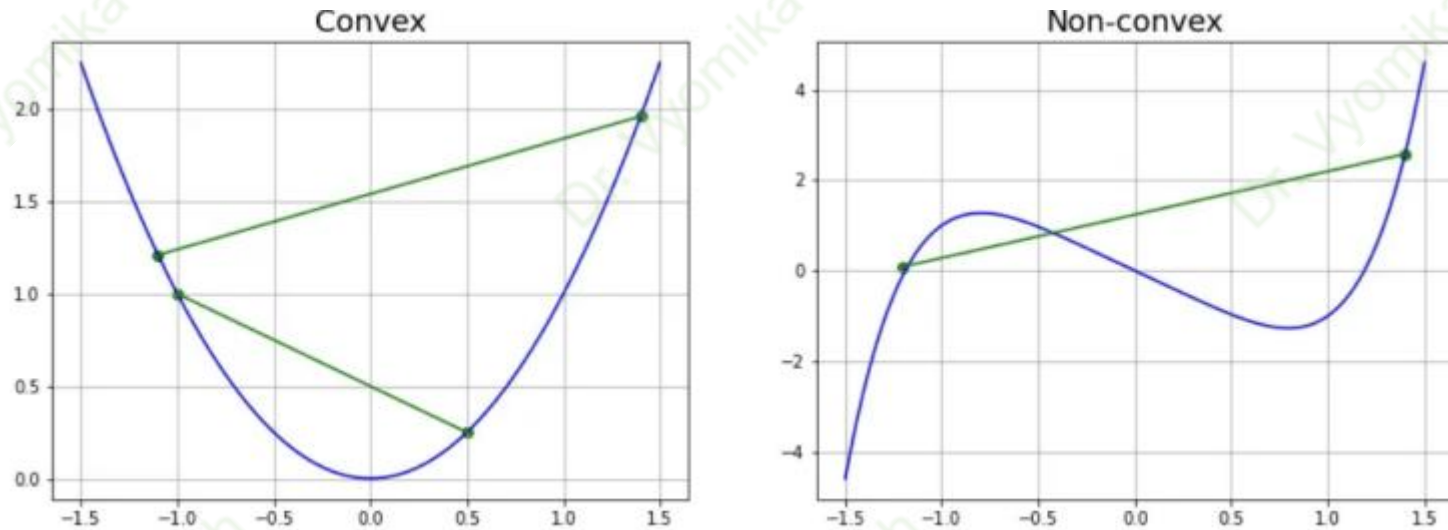


Figure: Examples of convex and non-convex functions.[Kwiatkowski 2022]

CSN 342

Deep Learning

Gradient Descent: Requirements

Exception: A special case is where the function has a **saddle point**. At the saddle point, the function has gradient as zero, but the double derivative is also zero at that point. This defines that the point is a saddle and hence not a global minimum.

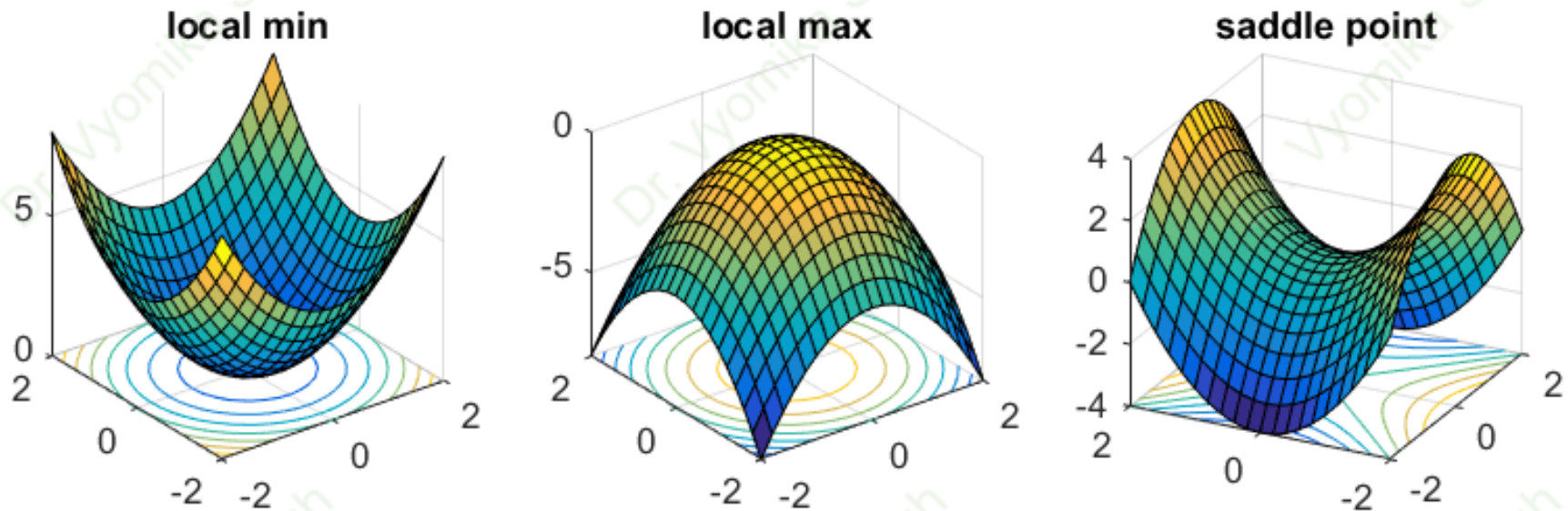


Figure: Examples of convex and non-convex functions.[Kwiatkowski 2022]

CSN 342

Deep Learning

Gradient Descent: Requirements

Therefore, the **gradient descent algorithm** takes 5 parameters as its basic requirement:

1. Initial Point (w_0 , b_0)
2. Gradient Function
3. Learning Rate
4. Number of iterations (n)
5. Tolerance: helps to give an end point/ stop to an algorithm.

CSN 342

Deep Learning

Numerical: how to solve

While performing the algorithm of gradient descent, the machine iteratively calculates the next point it has to reach by using the gradient at the current position, and subtracting it from the parameter value by scaling it with a learning rate. The formula for the same looks like:

$$p(n+1) \text{ or } p(\text{new}) = p_n - l \nabla f(p_n) \text{ // Update rule}$$

where, l is the learning rate, p_n is the parameter to be optimized, and $\nabla f(p_n)$ depicts the gradient of the expected loss function.

CSN 342

Deep Learning

Numerical: how to solve

1. Take $f(x)$

2. Calculate first derivative, $f'(x)$:

It tells us about **slope**:

- A. $f'(x) > 0$: function increasing
- B. $f'(x) < 0$: function decreasing
- C. $f'(x) = 0$: possible max/min

So **first derivative** talks about up/down behavior.

3. Calculate second derivative, $f''(x)$:

tells us about **curvature (shape)**:

- A. $f''(x) > 0$: curve is **cup-shaped (U)** i.e. convex
- B. $f''(x) < 0$: curve is **cap-shaped (∩)** i.e. concave

So **second derivative** = bending of the curve.

- 1. $f'(x) > 0$: **STRICTLY CONVEX**
- 2. $f'(x) < 0$: **CONCAVE**
- 3. $f'(x) = 0$: **LINEAR, FLAT**

Why do we solve $f''(x) = 0$?

Because:

- 1. Shape can change **only where curvature becomes zero or undefined**
- 2. If $f''(x)$ changes sign, it must pass through 0

Candidates for shape change:
 $f''(x)=0$

CSN 342

Deep Learning

Numerical: how to solve

Algorithm:

1. Given $f(x)$
2. Compute $f'(x)$
3. Compute $f''(x)$
4. Solve $f''(x)=0$ (and check undefined points)
5. Split number line into intervals
6. Check sign of $f''(x)$ in each interval:
 1. $f''(x) > 0$: convex
 2. $f''(x) < 0$: concave
7. If sign changes: not globally convex

CSN 342

Deep Learning

Numerical: how to solve

Q1. $f(x) = x^2 - x + 3$
 $f'(x) = 2x - 1 + 0$
 $f'(x) = 2x - 1$
 $f''(x) = 2$

Q2. $f(x) = x^4 - 2x^3 + 2$
 $f'(x) = 4x^3 - 6x^2 + 0$
 $= 4x^3 - 6x^2$
 $f''(x) = 12x^2 - 12x$
 $= 12x(x-1)$ { factored }

now let's see critical points for this value.

1. CRITICAL POINTS $\longrightarrow$ solving $f''(x) = 0$;

to see where
curve is LINEAR

solving $f''(x)$
normally algebraically

$\therefore$ for $12x(x-1)$; at $x=0$ and 1
value of $f''(x) = 0$

$-\infty$ to 0 0 to 1 1 to ∞

now for $12x(x-1)$

this
will
always be +ve

this product
depends on
 x +ve/-ve
 $(x-1)$ +ve/-ve

CSN 342

Deep Learning

Numerical: how to solve

for $(-\infty, 0)$: $f''(x) > 0$; so here
it is
CONVEX

for $(0, 1)$: $f''(x) < 0 \rightarrow$ concave
[value]

for $(1, \infty)$: $f''(x) > 0 \rightarrow$ again convex

$\therefore$ it's convex in $(-\infty, 0)$ and $(1, \infty)$

CSN 342

Deep Learning

Numerical: how to solve; questions with loss/cost functions

Use this whenever the loss depends on two parameters w and b .

Step 1: Write the model (Example: $y_{\text{pred}} = wx + by$)

Step 2: Write the loss function

For one data point (MSE) : $L(w, b) =$ whatever is given in the question

Step 3: Identify the parameters w, b .

Step 4: Compute partial derivatives (gradients); Because there are two parameters, compute two derivatives: $\partial L / \partial w$ and $\partial L / \partial b$

Rule to remember: one parameter means one partial derivative

Step 5: Substitute numerical values

Step 6: Update parameters (Gradient Descent rule) compute learning/update rate

We calculate two derivatives because the loss function depends on two parameters w and b , and gradient descent needs the derivative with respect to each parameter to update them.

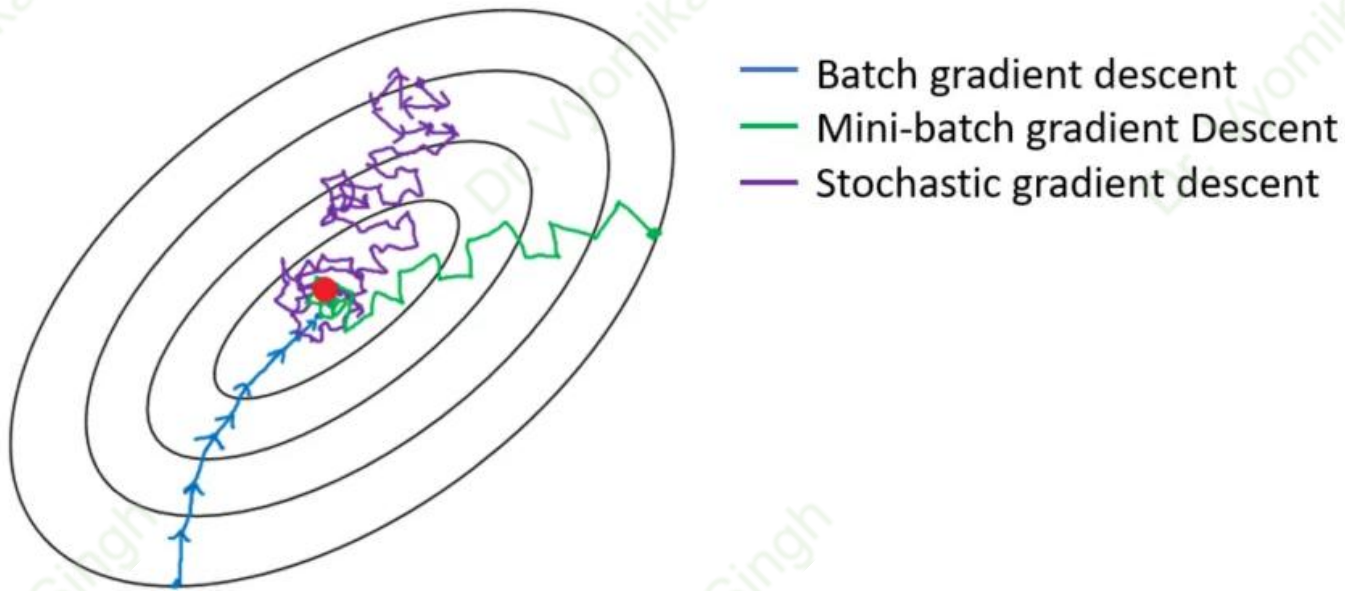
CSN 342

Deep Learning

Gradient Descent: Types

There are broadly **three types** of Gradient Descent Algorithms:

1. Batch Gradient Descent (**BGD**)
2. Stochastic Gradient Descent (**SGD**)
3. Mini-Batch Gradient Descent (**MBGD**)



CSN 342

Deep Learning

Gradient Descent: BGD

BGD computes the gradient based on the average of the gradients of all data samples in the training set.

1. Therefore, for each iteration, **the gradient has to be calculated for the entire dataset, making BGD a computationally expensive process for huge datasets.**
2. However, it also gives BGD the advantage of being more stable and avoiding overfitting compared to SGD.
3. The learning parameter has to be **chosen carefully**, so that with each iteration the step size doesn't get bigger or diverge from the minima.
4. BGD is usually used in simpler models with lesser data.
5. It is widely used in linear regression and logistic regression, not so much in deep learning where the training set is typically large and the models have many parameters.

CSN 342

Deep Learning

Gradient Descent: SGD

SGD computes the gradient for **only one random sample at each iteration**.

1. This property of SGD helps in it being faster and efficient as it does not have to process all the data in each of its iterations.
2. However, the randomness of SGD contributes to the fact that it can in some cases give the sub-optimal solutions/local minima as the result rather than the global minimum.
3. One of the techniques to overcome this fault is to decrease the learning rate of the model over time, which helps in reducing the updates in the parameter with each iteration.
4. SGD also has its variants, like Mini-Batch SGD, where the Gradient descent is done for a random subset of data, and Momentum SGD, where a term is added to the gradient update to help with the optimization and avoiding getting stuck at a local minima.
5. SGD is majorly used in Deep Learning and has found applications in classification, regression, and neural machine translation.

CSN 342

Deep Learning

Gradient Descent: MBGD

MBGD is a combination of SGD and BGD.

1. MBGD, in comparison to BGD, computed the gradient over a subset of the data, called the mini-batch.
2. This helps in **computing faster than BGD and avoiding overfitting as compared to SGD.**
3. The mini-batch size is a trade-off between speed and stability, with smaller sizes leading to faster convergence but increased variability in the optimization, and larger sizes leading to more stable convergence but slower processing times.
4. The learning rate, which determines the size of the parameter update at each iteration, must be carefully tuned to ensure that the optimization converges to the minimum and does not oscillate or diverge.
5. Mini-Batch Gradient Descent is widely used in deep learning and has been applied to a variety of tasks, including classification, regression, and neural machine translation

CSN 342

Deep Learning

Gradient Descent: Comparison

Batch Gradient Descent

- Entire dataset for updation
- Cost function reduces smoothly
- Computation cost is very high

Stochastic Gradient Descent (SGD)

- Single observation for updation
- Lot of variations in cost function
- Computation time is more

Mini-Batch Gradient Descent

- Subset of data for updation
- Smoother cost function as compared to SGD
- Computation time is lesser than SGD
- Computation cost is lesser than Batch Gradient Descent

CSN 342

Deep Learning

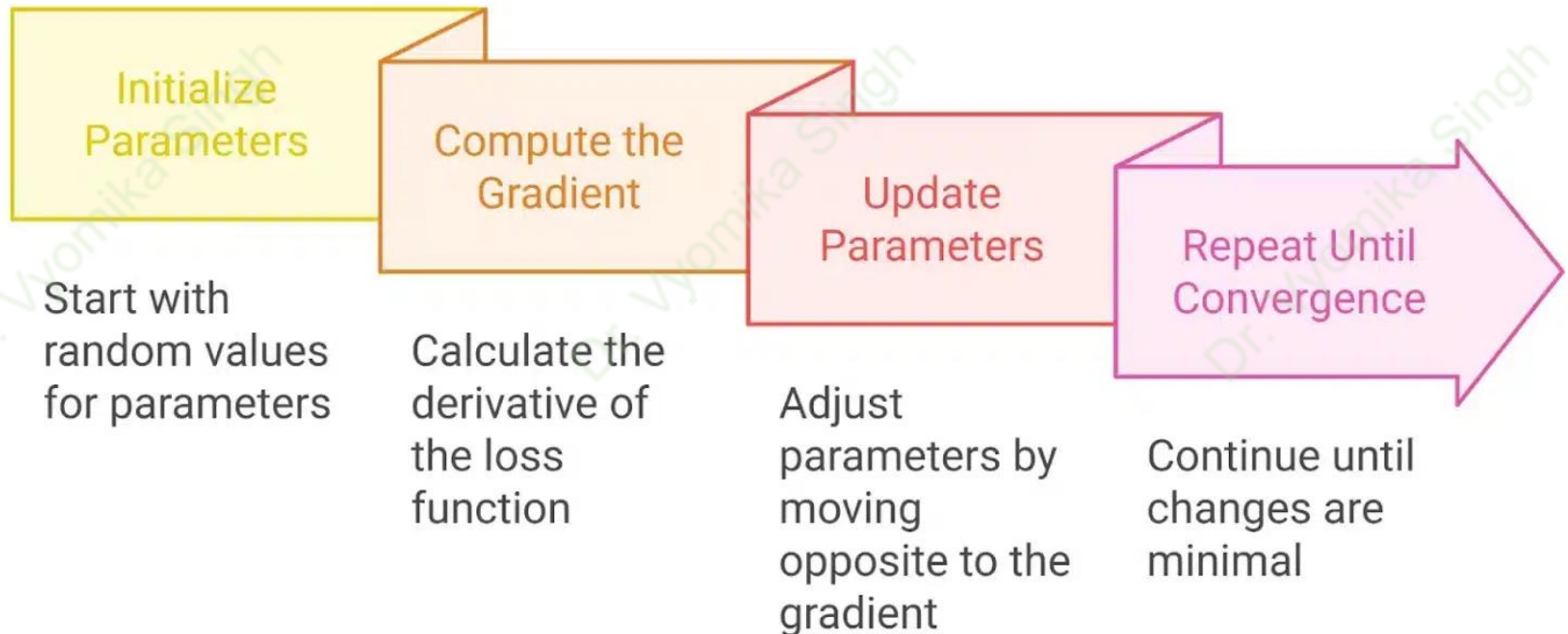
Gradient Descent: Comparison

PARAMETERS	BATCH GD ALGORITHM	MINI BATCH ALGORITHM	STOCHASTIC GD ALGORITHM
ACCURACY	HIGH	MODERATE	LOW
TIME CONSUMING	MORE	MODERATE	LESS

CSN 342

Deep Learning

Gradient Descent: Algorithm



CSN 342

Deep Learning

Gradient Descent: Algorithm

ALGO:

Initialize parameters θ

Repeat:

- Compute predictions using θ

- Compute loss $L(\theta)$

- Compute gradients $\partial L / \partial \theta_i$ for all parameters

- For each parameter θ_i :

- $\theta_i = \theta_i - \eta * (\partial L / \partial \theta_i)$

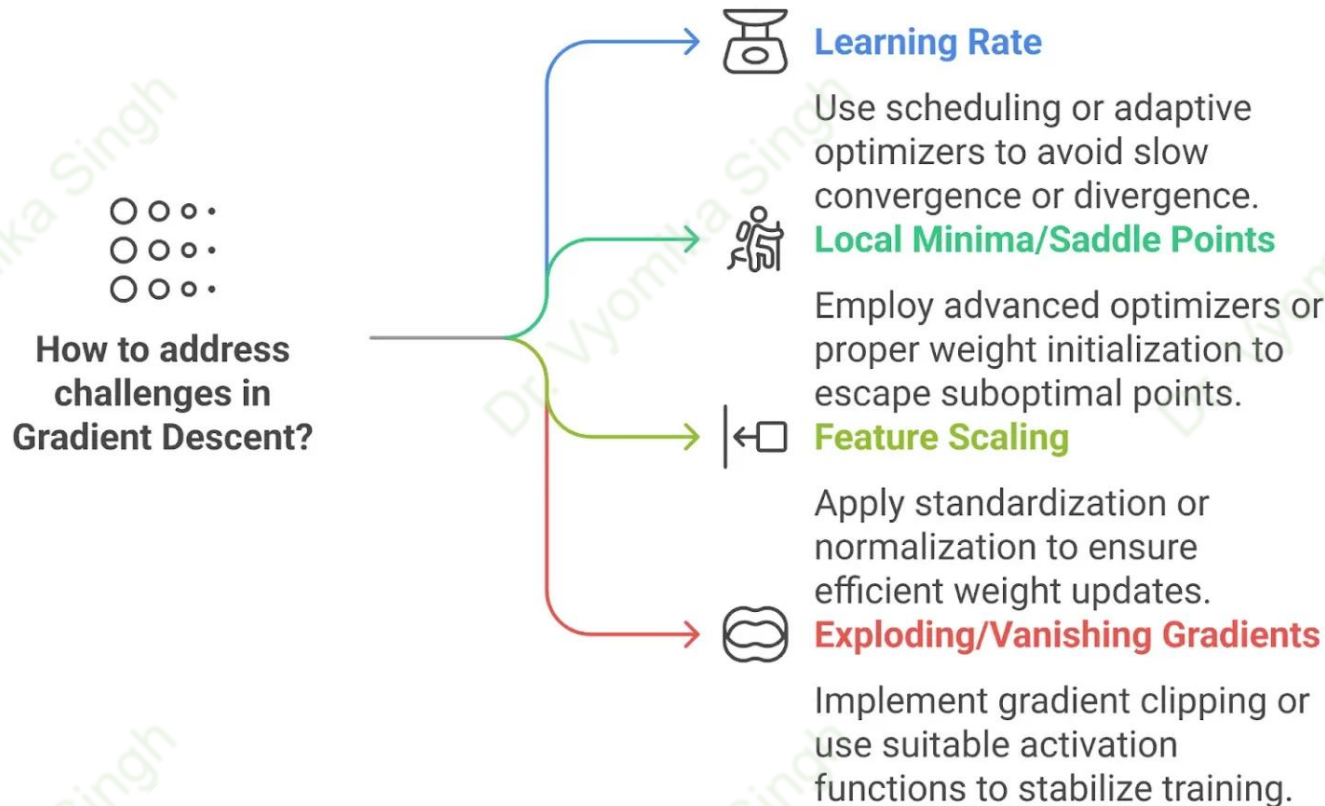
Until convergence

Return θ

CSN 342

Deep Learning

Gradient Descent: Challenges



CSN 342

Deep Learning

Gradient Descent: Challenges

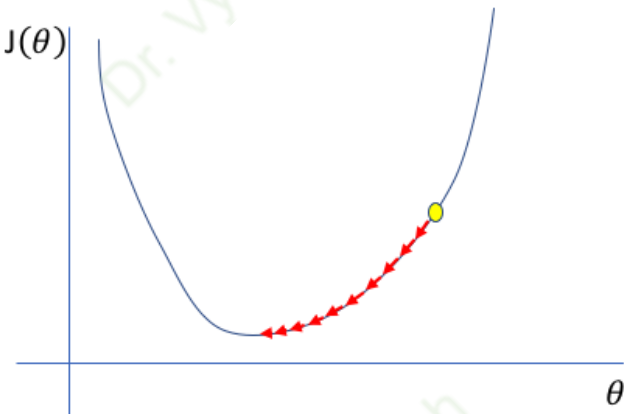
1. The **learning rate (α)** determines how large a step the algorithm takes toward the optimal solution.

If too small: Convergence is extremely slow, requiring many iterations to reach the minimum.

If too large: The algorithm may overshoot the minimum or diverge instead of converging.

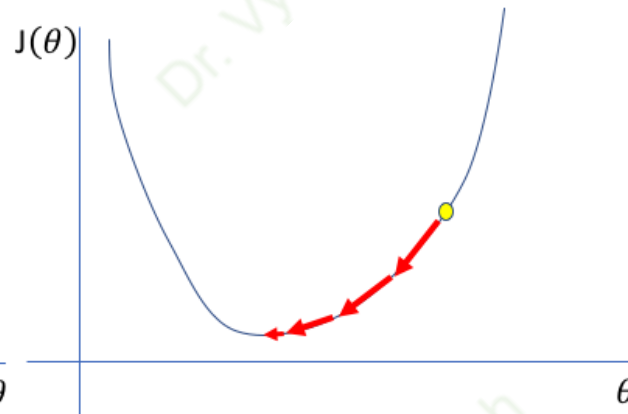
Solution: Techniques like learning rate scheduling, adaptive optimizers (e.g., Adam, RMSprop), or manual tuning help in selecting the right learning rate.

Too low



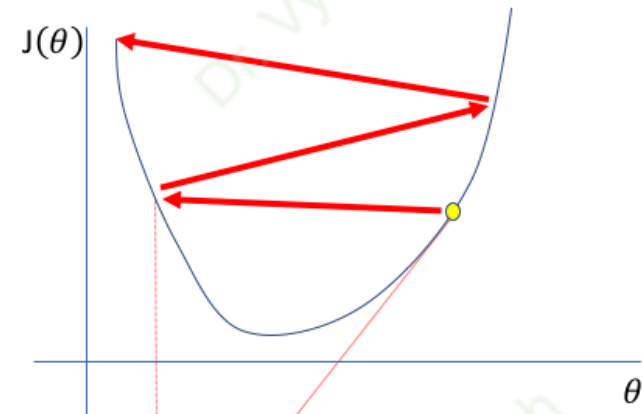
A small learning rate requires many updates before reaching the minimum point

Just right



The optimal learning rate swiftly reaches the minimum point

Too high

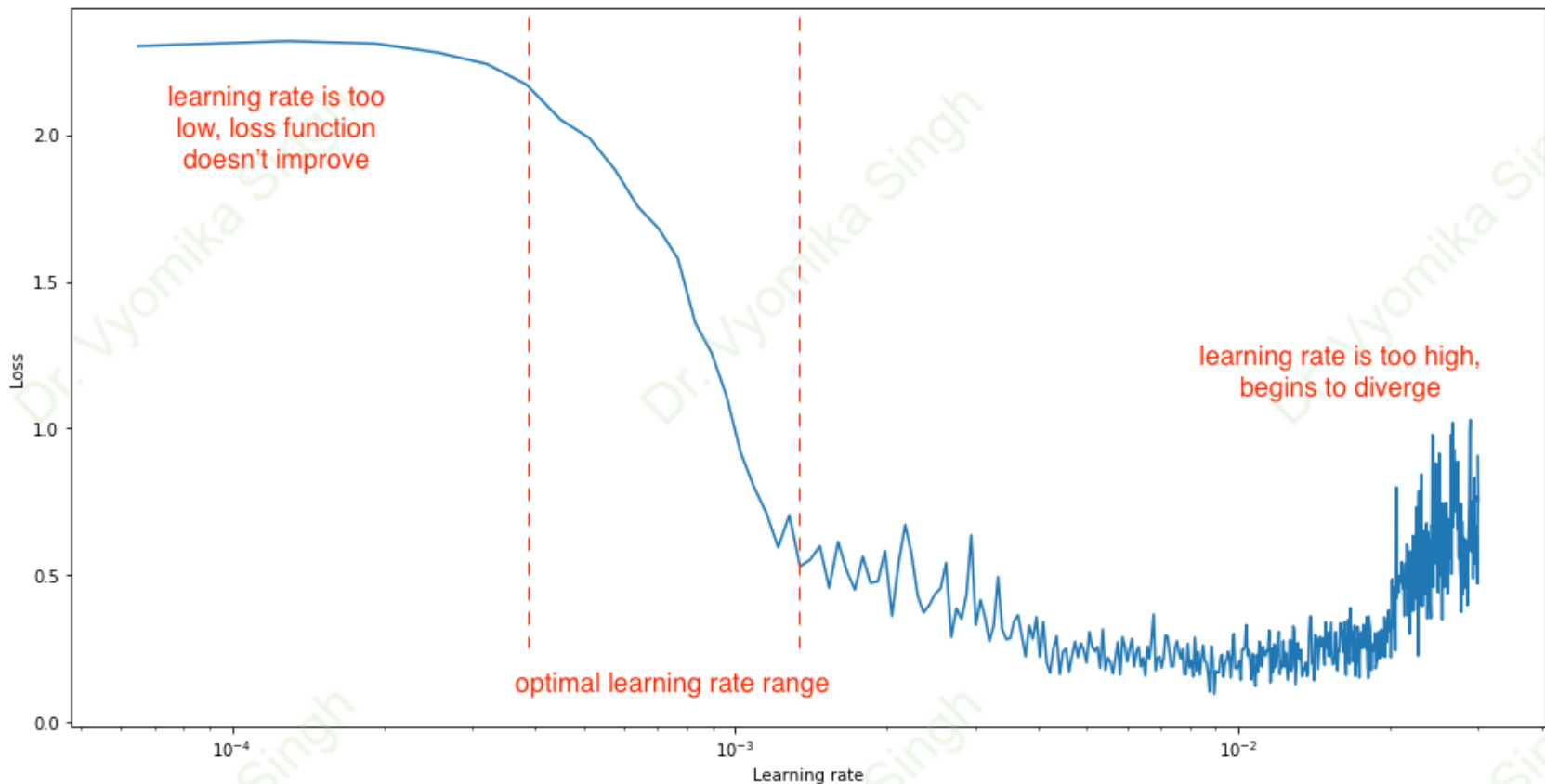


Too large of a learning rate causes drastic updates which lead to divergent behaviors

CSN 342

Deep Learning

Gradient Descent: Challenges



CSN 342

Deep Learning

Backpropagation and Chain Rule

Backpropagation is an algorithm that uses the **chain rule of calculus to efficiently calculate the gradient of the loss function in a neural network**. This gradient reveals how each weight and bias in the network affects the overall error, enabling the model to learn and adjust its parameters through optimization algorithms like gradient descent

Generally speaking, training neural networks with backpropagation entails the following steps:

1. ***A forward pass***, making predictions on training data.
2. ***A loss function*** measures the error of the model's predictions during that forward pass.
3. ***Backpropagation*** of error, or a ***backward pass***, to calculate the partial derivatives of the loss function.
4. ***Gradient descent***, to update model weights.

CSN 342

Deep Learning

Backpropagation and Chain Rule

Backpropagation is the cornerstone of training neural networks. It involves the iterative process of computing gradients and updating the network's parameters to minimize a chosen loss function. **At the core of backpropagation is the application of the chain rule to compute these gradients efficiently.**

1. Forward Pass:

The input data is passed through the network layer by layer. Each layer performs a weighted sum and applies an activation function to produce the output. These outputs are stored for later use in the backward pass.

2. Backward Pass (Backpropagation):

Starting from the output layer, the gradient of the loss function with respect to the output is computed. The chain rule is applied iteratively to compute the gradient of the loss function with respect to each layer's parameters and inputs. Gradients are propagated backward through the network.

CSN 342

Deep Learning

Backpropagation and Chain Rule

3. Parameter Update:

The computed gradients are used to update the network's parameters (weights and biases) through optimization algorithms like gradient descent. This process is repeated for a predefined number of iterations or until convergence.

CSN 342

Deep Learning

Backpropagation and Chain Rule

Let's break down how the chain rule is applied during the backpropagation process with a simple example. Consider a single-layer neural network with a linear transformation followed by a sigmoid activation function.

1. Forward Pass:

The input data x is transformed through a linear transformation: $z = Wx + b$, where W is the weight matrix and b is the bias. The output y is obtained by applying the sigmoid activation function: $y = \sigma(z)$.

2. Backward Pass (Backpropagation):

- Compute the gradient of the loss (L) with respect to the output (y): dy/dL
- Apply the chain rule to find the gradient of the loss with respect to the input z :

$$\frac{dL}{dz} = \frac{dL}{dy} \cdot \frac{dy}{dz}$$

- The derivative of the sigmoid function is $\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z))$.
- So, $\frac{dy}{dz} = \sigma(z)(1 - \sigma(z))$.

CSN 342

Deep Learning

Backpropagation and Chain Rule

c. **Compute the gradient** of the loss with respect to the parameters (W and b):

$$\begin{aligned} \bullet \frac{dL}{dW} &= \frac{dL}{dz} \cdot \frac{dz}{dW} = x \cdot \frac{dL}{dz} \\ \bullet \frac{dL}{db} &= \frac{dL}{dz} \cdot \frac{dz}{db} = \frac{dL}{dz} \end{aligned}$$

3. **Parameter Update:** Update the parameters W and b using the computed gradients and an optimization algorithm.

This example illustrates how the chain rule is used to efficiently compute the gradients necessary for parameter updates. The process is analogous for deep neural networks, with the chain rule applied layer by layer during the backpropagation process.

CSN 342

Deep Learning

Backpropagation and Chain Rule

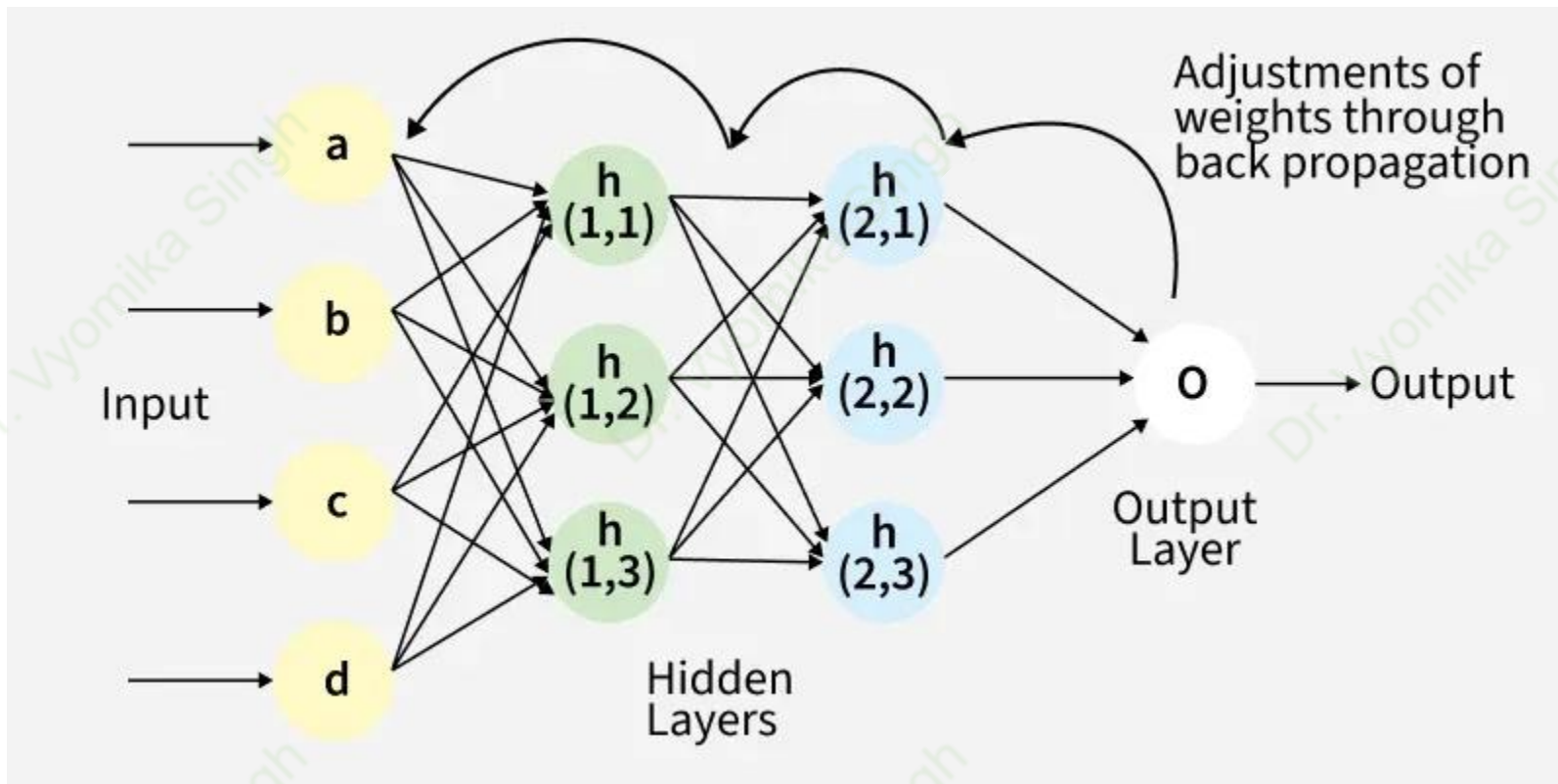


Fig A simple illustration of how the backpropagation works by adjustments of weights

CSN 342

Deep Learning

Backpropagation and Chain Rule

Back Propagation plays a critical role in how neural networks improve over time. Here's why:

1. **Efficient Weight Update:** It computes the gradient of the loss function with respect to each weight using the chain rule making it possible to update weights efficiently.
2. **Scalability:** The Back Propagation algorithm scales well to networks with multiple layers and complex architectures making deep learning feasible.
3. **Automated Learning:** With Back Propagation the learning process becomes automated and the model can adjust itself to optimize its performance.

CSN 342

Deep Learning

Chain Rule

The chain rule of calculus is a foundational concept that allows us to find the derivative of a composite function. In the context of deep learning, a neural network can be seen as a composition of many functions, with each layer representing a transformation of the input data. The chain rule enables us to calculate the gradient of the network's output with respect to its parameters, which is essential for updating the weights and biases during the training process. The chain rule states that if we have a composite function $f(g(x))$, then the derivative of this function with respect to x is given by:

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg} \cdot \frac{dg}{dx}$$

In the context of deep learning, f represents the final output of the neural network, g represents the intermediate activations at each layer, and x represents the input data.

CSN 342

Deep Learning

ERM: Empirical Risk Minimization

Empirical Risk Minimization (ERM) is a fundamental concept in machine learning used for training models by minimizing the average loss over a given dataset. It provides a formal framework to select the model parameters that best fit the training data.

- We have a training dataset $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, where:
 - x_i are input features.
 - y_i are true outputs/labels.
- We want to train a model $f(x; \theta)$ that predicts y for a given x , where θ represents the model's parameters.

CSN 342

Deep Learning

ERM: Empirical Risk Minimization

Loss Function:

- Define a loss function $\mathcal{L}(y, f(x; \theta))$ that measures how far the predicted value $f(x; \theta)$ is from the true value y .

Empirical Risk:

- The empirical risk is the average loss across all n training examples:

$$R_{emp}(\theta) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f(x_i; \theta)).$$

Goal:

- Find the parameter θ that minimizes the empirical risk:

$$\theta^* = \arg \min_{\theta} R_{emp}(\theta).$$

CSN 342

Deep Learning

ERM: Empirical Risk Minimization

Key Aspects of ERM in Deep Learning:

- **Goal:** The primary objective is to select a model (hypothesis) that has the lowest possible error on the training data, assuming this will lead to good performance on new data.
- **Loss Function:** A critical component that quantifies the difference between predictions and actual targets (e.g., Mean Squared Error, Cross-Entropy).
- **Optimization:** Since finding the absolute minimum is difficult, iterative methods like Stochastic Gradient Descent (SGD) are typically used to minimize the empirical risk.
- **Limitations:** Pure ERM often leads to overfitting, where the model learns the training data too well (including noise) and performs poorly on new data.
- **Regularization:** To counter overfitting, regularization techniques (like L1/L2 regularization) are often added to ERM to constrain model complexity.

CSN 342

Deep Learning

ERM: Empirical Risk Minimization

1. **Overfitting:** Minimizing the empirical risk alone may lead to overfitting, where the model performs well the training data but poorly on unseen data. Regularization techniques, such as or regularization, are often used to mitigate this.
2. **Generalization:** ERM assumes the training data is representative of the entire data distribution. Generalization error measures the difference between the empirical risk and the true risk.
3. **Optimization:** Minimizing the empirical risk can be computationally intensive for complex models or large datasets.

CSN 342

Deep Learning

ERM: Empirical Risk Minimization

Extensions of ERM:

1. Regularized ERM:

- Adds a penalty term to the risk to control model complexity:

$$R_{reg}(\theta) = R_{emp}(\theta) + \lambda \Omega(\theta)$$

where $\Omega(\theta)$ is the regularization term, and λ controls the strength of regularization.

2. Stochastic ERM:

- Instead of using the entire dataset, stochastic gradient descent (SGD) is used to optimize the risk incrementally using mini-batches.

CSN 342

Deep Learning

Given: $y = wx + b$ // Model

$$(x, y) = (2, 5)$$

$$L(w, b) = \frac{1}{2} \times (y_{\text{predicted}} - y)^2$$

$$\text{where } y_{\text{predicted}} = w \cdot x + b$$

$$w = 0, b = 0, \eta = 0.1$$

To do: perform one iteration of gradient descent to update w and b ; show the gradients, updated values of w and b , and the loss after update

① let's solve with values also known as forward pass:

$$y_{\text{pred}} = wx + b = (0)(2) + (0) = 0$$

② loss computation; $L = \frac{1}{2} \times (0 - 5)^2 = 12.5$

now ③ we have to calculate gradients with both w and b

$$L = \frac{1}{2} (wx + b - y)^2$$

$$\frac{\partial L}{\partial w} = (wx + b - y) \cdot x$$

$$= (0 - 5) \cdot 2 = -10$$

solved by chain rule

$$u = wx + b - y; L = \frac{1}{2} u^2$$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial u} \cdot \frac{\partial u}{\partial w}$$

$$\therefore \frac{\partial L}{\partial u} = \frac{1}{2} \times 2u = u$$

$$\frac{\partial u}{\partial w} = x$$

$$(wx + b - y) \cdot x$$

CSN 342

Deep Learning

Consider the following simple linear regression model:

$$y = w \cdot x + b$$

You are given a dataset with a single data point $(x, y) = (2, 5)$. The loss function is the Mean Squared Error (MSE), defined as:

$$L(w, b) = \frac{1}{2} \times (y_{\text{predicted}} - y)^2$$

Where $y_{\text{predicted}} = w \cdot x + b$.

Given the initial values $w = 0$, $b = 0$, and a learning rate $\eta = 0.1$, perform one iteration of gradient descent to update w and b . Show the gradients, the updated values of w and b , and the loss after the update.

CSN 342

Deep Learning

Given:

- Data point: $(x, y) = (2, 5)$
- Initial values: $w = 0, b = 0$
- Learning rate: $\eta = 0.1$
- Loss function (MSE):

$$L(w, b) = \frac{1}{2} \times (y_{\text{predicted}} - y)^2$$

where $y_{\text{predicted}} = w \cdot x + b$.

Step 1: Compute $y_{\text{predicted}}$

Using the initial values of w and b :

$$y_{\text{predicted}} = (0 \cdot 2) + 0 = 0$$

Step 2: Compute the Loss

$$L(w, b) = \frac{1}{2} \times (0 - 5)^2 = \frac{1}{2} \times 25 = 12.5$$

Step 3: Compute the Gradients

The gradients of the loss function with respect to w and b are:

1. Gradient with respect to w :

$$\frac{\partial L}{\partial w} = (y_{\text{predicted}} - y) \cdot x = (0 - 5) \cdot 2 = -10$$

2. Gradient with respect to b :

$$\frac{\partial L}{\partial b} = (y_{\text{predicted}} - y) = 0 - 5 = -5$$

Step 4: Update w and b

Using the gradient descent update rules:

$$w \leftarrow w - \eta \times \frac{\partial L}{\partial w}$$

$$b \leftarrow b - \eta \times \frac{\partial L}{\partial b}$$

Substituting the values:

$$w \leftarrow 0 - 0.1 \times (-10) = 0 + 1 = 1$$

$$b \leftarrow 0 - 0.1 \times (-5) = 0 + 0.5 = 0.5$$

CSN 342

Deep Learning

Step 5: Compute the New Loss

Now, compute the loss with updated w and b :

$$y_{\text{predicted}} = (1 \cdot 2) + 0.5 = 2.5$$

$$L(w, b) = \frac{1}{2} \times (2.5 - 5)^2 = \frac{1}{2} \times (-2.5)^2 = \frac{1}{2} \times 6.25 = 3.125$$

Final Updated Values:

- $w = 1$
- $b = 0.5$
- New Loss: 3.125

This completes one iteration of gradient descent.

CSN 342

Deep Learning

Numerical

Suppose you have the following dataset for a simple regression problem:

x	y
1	3
2	5
3	7
4	9

The model is $f(x; w) = wx$, where w is the weight parameter. Use the Mean Squared Error (MSE) loss function defined as:

$$L(w) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; w))^2$$

Find the value of w that minimizes the empirical risk.

Deep Learning

$$L(w) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; w))^2$$

goal is to find the value of w that minimizes the empirical risk

we have 4 iterations; $\therefore n=4$ & $f(x; w) = wx$

$$L(w) = \frac{1}{4} \sum_{i=1}^4 (y_i - wx_i)^2$$

now we will differentiate w.r.t w by minimising $L(w) \rightarrow 0$ [or $\frac{dL}{dw} / \frac{\partial L}{\partial w} \rightarrow 0$]

$$\frac{dL}{dw} = \frac{d}{dw} \left(\frac{1}{4} \sum_{i=1}^4 (y_i - wx_i)^2 \right)$$

now, this is a composite fⁿ; $\therefore z = y_i - wx_i$

$$\text{meaning } \frac{d}{dw} (z^2) = 2z \cdot \frac{dz}{dw}$$

x_i, y_i const,
 w variable

$$\therefore \frac{dz}{dw} = \frac{d}{dw} (y_i - wx_i) = -x_i$$

chain rule result

$$\frac{d}{dw} (y_i - wx_i)^2 = 2 (y_i - wx_i) (-x_i)$$

$$\text{Now } \frac{dL}{dw} = \frac{1}{4} \sum_{i=1}^4 2 (y_i - wx_i) (-x_i)$$

simplify:

$$\frac{dL}{dw} = -\frac{2}{4} \sum_{i=1}^4 x_i (y_i - wx_i)$$

$$\frac{dL}{dw} = -\frac{1}{2} \sum_{i=1}^4 x_i (y_i - wx_i)$$

when we calculate minimum; constant can be ignored

$$\text{Now } \frac{dL}{dw} \rightarrow 0 \quad [\text{as we want to calculate minimal}]$$

$$0 = \sum_{i=1}^4 x_i (y_i - wx_i)$$

CSN 342

Deep Learning

$$0 = \sum_{i=1}^4 x_i y_i - w \sum_{i=1}^4 x_i^2$$

$$w \sum_{i=1}^4 x_i^2 = \sum_{i=1}^4 x_i y_i$$

so w can be

$$\frac{\sum_{i=1}^4 x_i y_i}{\sum_{i=1}^4 x_i^2}$$

$$w = \frac{(1)(3) + (2)(5) + (3)(7) + 4(9)}{1^2 + 2^2 + 3^2 + 4^2} = \frac{70}{30}$$

$$= 7/3 \text{ Or } 2.33 \text{ [approx]}$$